

Exemplar-based and Kernel Methods in Machine Learning

Sunita Sarawagi

February 18, 2026

Introduction to Exemplar/Kernel Methods

- A nonparametric method where the prediction function does not have a fixed functional form but instead estimated directly from the data.
- Given training data $D = \{(\mathbf{x}^i, y^i) : i = 1 \dots N\}$ are observed N points of an unknown function $f(\mathbf{x})$, make prediction purely based on **similarity** of a test point $\mathbf{x}^*$ to training points. Need to keep around training data.
- Similarity measured as a kernel function $k(\mathbf{x}^*, \mathbf{x}^i)$ between any two points.
- Equivalently dissimilarity between two points specified in terms of a distance function $d(\mathbf{x}^*, \mathbf{x}^i)$
- No need to know coordinates of points in any fixed dimensional space.

Nearest neighbor classifier

Simplest kind of classifier. (hyper-parameter $R \equiv \#$ of examples considered.)

For each x^* ,

find the R points from D that are closest to x^*

$$\text{Nbr}(x^*) = \underset{1 \leq i \leq N}{\operatorname{argmin}_R} d(x^*, x^i)$$

for each class $P(\hat{y} = c | x^*) = \frac{\# \text{ of examples in } \text{Nbr}(x^*) \text{ with label } c}{R}$

$$\hat{y} = \underset{1 \leq c \leq k}{\operatorname{argmax}} P(\hat{y} = c | x^*)$$

Example

```
https://drive.google.com/file/d/  
1LihEFIe5fyFbWPS342PA9NZdy2Kxjkvy/view?usp=sharing
```

Limitations

1. KNN does not work well for high-dimensional data. As dimension increases, most points become equally far from each other.
2. Need to design a meaningful distance measure for KNN to be useful. Lots of work on metric learning.
3. Need to store the training data, and retrieve during deployment. Slow.

Application: Few-shot classification

Classification with dynamically changing class labels. Do not need to decide on the set of classes during training time.

For each test label x^* , user provides n examples over a dynamically decided set of class labels.

$$\text{Nbr}(x^*) = \{ (x^i, y^i) - - (x^R, y^R) \}$$

where each $y^i \in$ test-time selected class labels

Learning distance functions

$$d_{\theta}(x^i, x^j)$$

Many methods have been proposed.

Assume we have supervision on pairs of objects that are similar and dissimilar.

Learn a distance function that brings together similar objects and keeps apart dissimilar objects.

A starting loss function could be

$$\min_{\theta} \sum_{j=1}^N \sum_{i=1}^N \left\{ L(\mathbf{x}^i, \mathbf{x}^j) = \delta(y_i = y_j) d_{\theta}(\mathbf{x}^i, \mathbf{x}^j) + \delta(y_i \neq y_j) \max(0, m - d_{\theta}(\mathbf{x}^i, \mathbf{x}^j)) \right\}$$

$\min_{\theta} \sum_i \sum_j \delta(y_i = y_j) d_{\theta}(x^i, x^j) + \delta(y_i \neq y_j) [-d(x^i, x^j)]$

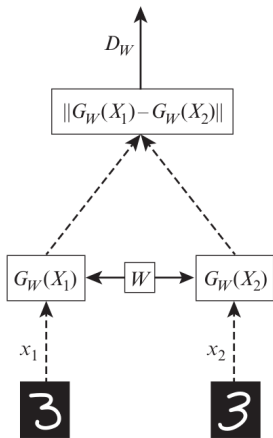
Many different models proposed in traditional and deep learning methods for representing distances.

A limitation of this objective is that we only care about bringing together examples of the same class, not how far apart examples of different classes are.

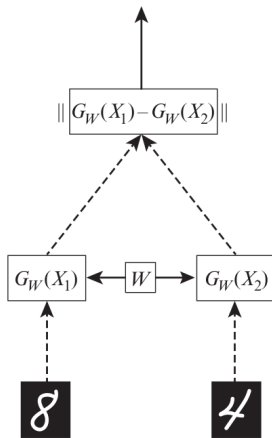
Siamese networks for learning distance functions

The Siamese Network

Minimize the distance



Maximize the distance



$G_W(x) \Rightarrow \mathbb{R}^m$; $x \in \mathbb{R}^d$
embedding function